Stewardex

TECHNICAL SPECIFICATION

Stewardex Mobile Application

Backend Developer Guide

Release	Phase 1
Date	15 June 2026
Status	For review

Stewardex — Charity Governance & Compliance Suite

CONFIDENTIAL

1. What Already Works (No Change Needed)

The mobile app reuses the existing REST API and Socket.IO server as-is:

- **Auth & MFA** — `/auth/login`, `/auth/otp/*`, `/auth/mfa/*`, `/auth/forgot-password`, `/auth/reset-password`, `/auth/me/permissions`, and TOTP under `/platform/me/profile/mfa/totp/*`.
- **Tenant resolution** — `authAndResolveTenant` keys off the `x-org-id` header (already supported); the mobile client sends it on every request.
- **All feature endpoints** — calendar, meetings, approvals, chat, policies, risks, COI, complaints, expenses, billing, support tickets, profile, notifications.
- **Real-time** — the `chatSocketService` rooms/events (`chat:*`, presence) work unchanged for a mobile socket client authenticating via the handshake JWT.
- **Stripe billing** — checkout/portal return URLs work with an in-app browser; existing webhooks are unaffected.
- **File upload** — S3 multipart upload + signed URLs (expenses, profile picture, chat attachments) work from mobile as-is.

CORS already permits the API to be called from non-browser clients; confirm the mobile origin/`null` origin is allowed (native apps don't send a browser `Origin` for REST, but the Socket.IO handshake should accept the app).

---

2. Required New Work: Native Push Notifications

Problem: The current push implementation is **Web Push (VAPID)** — `GET /notifications/push/public-key`, `POST /notifications/push/subscribe`, `POST /notifications/push/unsubscribe`. This delivers to **browser** service workers only and does **not** reach native iOS/Android apps.

Solution: Add a native push delivery path. Recommended: **Expo Push** (single API for both platforms) or **Firebase Cloud Messaging (FCM)** for Android + **APNs** for iOS. Expo is the least work and matches the recommended mobile stack.

2.1 New endpoints

Method	Endpoint	Body	Purpose
POST	`/api/v1/platform/notifications/push/register-device`	`{ token, platform: 'ios'|'android', deviceId, appVersion? }`	Store/upsert a device push token for the authenticated user.
POST	`/api/v1/platform/notifications/push/unregister-device`	`{ token }` *(or `{ deviceId }`)*	Remove a device token (called on logout / permission revoke).

Notes:

- **Upsert by** `token` (or `deviceId`) so re-registration on app launch is idempotent.
- Store under the user, scoped to the tenant DB, alongside the existing web-push subscription store. A token belongs to one `user_id` + `orgId`.
- Prune invalid tokens when the push provider reports `DeviceNotRegistered` / `Unregistered`.

2.2 New data model

Add a `DeviceToken` (or extend the existing push-subscription schema) in the **platform (tenant)** DB:

```
DeviceToken {
  user_id,            // owner
  token,              // Expo/FCM token or APNs token (unique)
  platform,           // 'ios' | 'android'
  device_id,          // stable per-install id
  app_version,        // optional, for debugging
  created_at,
  last_seen_at,
  disabled_at         // set when provider reports the token is dead
}
```

2.3 Sending push

Extend the existing notification-creation path so that **whenever an in-app** `Notification` **is created**, in addition to the web-push fan-out, the service also sends to the user's registered native device tokens.

- Add a `pushNotificationService.sendToUser(userId, orgId, { title, body, data })` that looks up `DeviceToken`s and calls the provider.
- `data` payload must carry `type`, `entity_type`, `entity_id` so the app can deep-link (e.g. `{ type: 'approval_pending', entity_type: 'approval', entity_id: '...' }`).
- Wire it into the same triggers that already create notifications: `approval_pending`, `workflow_approved`, `workflow_rejected`, `returned_for_resubmission`, `meeting_invitation`, `meeting_notes_added`, `chat_mention`, `ticket_assigned`.

- Respect existing notification preferences (don't push types the user has muted).

2.4 Config / secrets

- **Expo:** Expo access token (and APNs/FCM credentials uploaded to Expo) — set via `.env`.
- **Direct FCM/APNs:** FCM server key + APNs auth key (`.p8`), key id, team id, bundle id — via `.env`.
- Add a `expo-server-sdk` (or `firebase-admin` + `node-apn`) dependency.

---

3. Optional Conveniences (Nice-to-have, not blocking)

These reduce mobile-side complexity but are **not required** — the app can ship without them by aggregating client-side (as the web does).

3.1 `GET /platform/me/tasks` — unified task feed

"My Tasks" is currently composed client-side from `/approvals`, `/meetings`, and `/calendar`. A single endpoint that returns the merged, normalized list (with `type`, `title`, `due`, `category`, `entity_id`) would cut three round-trips to one and centralize the logic. Optional.

3.2 MFA challenge during login

The frontend expects MFA challenge codes (`MFA_REQUIRED | MFA_INVALID | MFA_EXPIRED`) on `/auth/login` for TOTP users. Confirm the login flow returns these codes for TOTP-enabled accounts (it already does for the web MFA path). No change if already present.

3.3 Password change endpoint

Confirm there is a direct "change password while logged in" endpoint under `/platform/me/profile` (current password + new). If the only path today is the email reset flow, add a `POST /platform/me/profile/password` for a smoother in-app experience. Verify before building.

3.4 Lightweight list payloads (optional optimization)

Mobile list screens (risks, complaints, policies) don't need full documents. If payloads are large, consider a `?fields=` / summary mode. Only pursue if profiling shows it matters.

---

4. Things to Verify / Watch

- **CORS & Socket.IO handshake** accept the mobile client (token in `handshake.auth.token`).
- **Signed S3 URL TTL** is long enough for a mobile user to open a document/invoice (and is re-fetchable).
- **Stripe checkout/portal return URLs** support a deep-link / universal-link scheme so the app can resume after the in-app browser closes; otherwise the existing web return URL + a "done" screen is acceptable.
- **Notification preferences** are honoured by the new native push path (don't double-notify or push muted types).

- **Token cleanup on logout** — the app calls `unregister-device`; also prune dead tokens reported by the provider.

5. Summary of Backend Deliverables

#	Item	Type	Priority
1	`POST /notifications/push/register-device`	New endpoint	**Required**
2	`POST /notifications/push/unregister-device`	New endpoint	**Required**
3	`DeviceToken` schema (platform DB)	New model	**Required**
4	Native push send (Expo or FCM+APNs) wired into notification triggers	New service	**Required**
5	Push provider credentials in `.env` + SDK dependency	Config	**Required**
6	`GET /me/tasks` unified feed	New endpoint	Optional
7	`POST /me/profile/password` (if not present)	New endpoint	Verify / maybe
8	Confirm MFA challenge codes on `/auth/login`	Verify	Verify
9	Confirm CORS / Socket.IO accepts native client	Verify	Verify

Everything else the mobile app needs is already served by the current API and Socket.IO server.